

Systematic Evaluation of Similarity Metrics for Retrieval, Reranking, and Completion in Retrieval Augmented Generation Systems

Harun Elkiran

Department of Computer Engineering
İstanbul Sabahattin Zaim University
Istanbul, Turkey
harun.elkiran@izu.edu.tr
0000-0002-5834-6210

Jawad Rasheed

Department of Computer Engineering
İstanbul Sabahattin Zaim University
Istanbul, Turkey
jawad.rasheed@izu.edu.tr
0000-0003-3761-1641

Abstract—Two of the major problems with large language models (LLMs) are hallucinations and out-of-context responses. To deal with these problems, Retrieval Augmented Generation (RAG) has emerged as a promising approach. It grounds the output of LLMs in external knowledge. The effectiveness of RAG pipelines depends on several factors, including the choice of similarity metric. This paper presents a systematic evaluation of a comprehensive RAG pipeline that utilizes the Milvus vector database with HNSW indexing techniques in conjunction with OpenAI's embedding models and GPT-based completion. We conducted a comparative analysis of three widely used similarity metrics —Cosine, Inner Product, and L2 —under identical conditions. Based on the results, it was observed that retrieval and reranking performance are highly sensitive to the similarity metrics. Cosine and Inner Product consistently achieve substantially higher recall ($R@10 = 0.9092$ – 0.925), Mean Reciprocal Rank ($MRR = 0.7806$ – 0.7930), and nDCG ($nDCG@10 = 0.8121$ – 0.8252) than L2. In contrast, completion stage metrics such as token usage, cost, and latency remain largely unaffected by the choice of metric. These results underscore the crucial role of retrieval similarity functions in determining RAG effectiveness.

Index Terms—Retrieval-Augmented Generation, Similarity Metrics, Reranking, Vector Databases, HNSW Indexing

I. INTRODUCTION

Hallucinations in large language models (LLMs) are frequently attributed to a variety of reasons [1]. While inherent model behavior leads to hallucinations, a major factor is a lack of meaningful context provided during generation. When the input context is insufficient or irrelevant to the query, LLMs are more likely to generate incorrect or manufactured results [2]. Retrieval-Augmented Generation (RAG) addresses this issue by providing the model with relevant document chunks from a knowledge base, guaranteeing that generated responses are factual and contextually suitable. Thus, RAG tackles both hallucination and relevance issues, increasing the credibility of LLM outcomes. Beyond hallucination, adapting general-purpose models to specialized domains is hindered by the scarcity of structured, domain-specific data. This mismatch complicates fine-tuning and impairs the ability of LLMs to understand and generate content within specialized domains.

Fine-tuning, however, remains critical, as it improves reading comprehension of domain texts and enhances a model's ability to utilize retrieved contexts effectively within RAG pipelines [3]. RAG has recently gained popularity for use in question and answer systems [4]. However, eliminating the epistemic uncertainty introduced by the use of large language models for retrieval and generation tasks is necessary to create a robust and dependable RAG pipeline [5]. Currently, reviewing RAG pipelines is a manual and iterative process that can be made more efficient by automating the evaluation process, comparable to automated test case generation [6].

In RAG systems, similarity score thresholds are used to determine the relevance of documents or objects by comparing their similarity scores, which indicate the degree of similarity between two items. It indicates the lowest score that an item must achieve to be considered relevant. People often discuss similarity thresholds when seeking to enhance retrieval performance. For instance, thresholds are adjusted to strike a balance between precision and recall, ensuring that the items identified are both relevant and distinct [7].

In this study, we systematically evaluated the impact of different similarity metrics, Cosine, Inner Product, and Euclidean (L2), within a standardized RAG pipeline. The main contributions of this work are as follows.

- In this study, we implemented a complete RAG pipeline with retrieval, reranking, and generation using Milvus and GPT models.
- We compared Cosine, Inner Product, and L2 distance metrics under identical conditions at all stages.
- We proposed an automated evaluation framework for RAG pipelines, minimizing manual assessment.

II. LITERATURE REVIEW

RAG is a powerful paradigm for enhancing the factuality and contextual relevance of large language model (LLM) output. A growing body of research has investigated the applications, evaluation, and limitations of RAG pipelines across multiple domains. [8] examined RAG pipelines in

the telecommunication domain, emphasizing the importance of similarity metrics such as factual correctness and answer similarity. This study proposed improvements to the RAGAS package for domain-specific evaluation. Additionally, [9] developed an enterprise knowledge management (EKM) assistant with RAG, which performed well in ROUGE, BLEU, and accuracy tests. In [10], the Overall Performance Index (OPI) was introduced for specialized reasoning tasks, which combines logical relation correctness and embedding similarity, and this method proved to be superior.

Some studies have proposed improvements in RAG architecture. For example, as in [11], retrieved chunks were reformatted into question-answer structures. It provides improved contextual grounding. As [12] introduced an inverted question matching approach QuIM-RAG. The approach proposed in [12] reduced hallucinations and improved retrieval accuracy using BERT Score and RAGAS metrics. Barochiya et al. [13] studied RAG pipelines for multimodal LLMs and found that GPT-4o-mini outperformed Gemini Pro in contextual correctness.

Several studies have focused on evaluation, as highlighted by [14], which underscored the high cost of human evaluations for RAG-based QA, underscoring the urgent need for automated similarity metrics. [15] proposed a framework (retEval) for clearer retriever assessment beyond standard recall and precision. Another framework (RAGEval) [16] creates scenario-specific dataset generation and evaluation metrics. RAG has also been applied in mission-critical applications such as healthcare [17], and regulation [18]. These applications require secure RAG evaluations as shown in [19]

Despite these advancements, most prior works either focus on architectural modifications. Still, few directly compare the impact of similarity metrics (e.g., cosine, inner product, L2) under identical experimental conditions across retrieval, reranking, and generation stages. This comparative gap motivates our study, which systematically analyzes these similarity measures within a controlled RAG pipeline, providing insights into their influence on end-to-end system performance.

III. METHODOLOGY

In this study, we implemented an RAG pipeline to evaluate question answering performance (see Fig. 1). We precomputed embeddings for all document chunks, indexed them in Milvus using the Hierarchical Navigable Small World (HNSW) algorithm, and evaluated queries with varying similarity metrics (cosine, inner product, L2). This study provides a comparison of similarity metrics (cosine, inner product, L2) while keeping all parameters constant. A detailed discussion of the pipeline is provided below.

A. Data Preprocessing

The first step in the RAG pipeline is data collection and preprocessing. In this study, we used a large-scale SQuAD corpus [20]. This corpus was systematically segmented into fixed-size document chunks. Document chunking is performed to enable the efficient and scalable retrieval of documents.

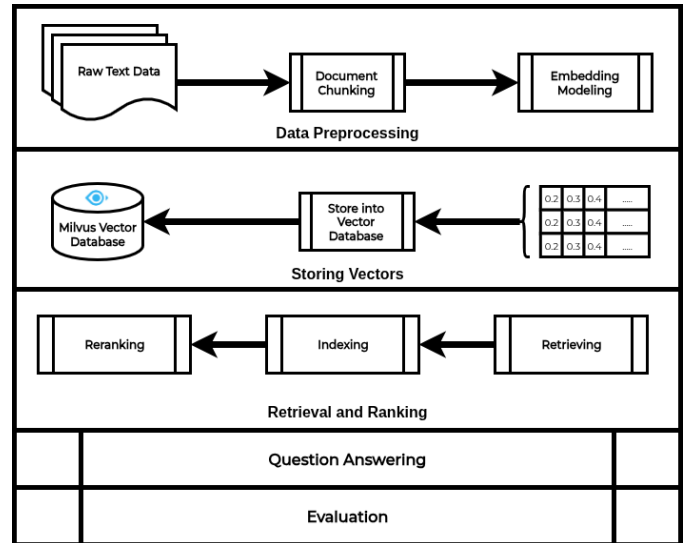


Fig. 1. Retrieval Augmented Generation (RAG) pipeline

Each chunk of the document is uniquely labeled with a doc_id and a chunk_id. With unique doc_id and chunk_id, precise traceability of the document is achieved, providing ease of access to ground-truth responses. Chunks were created with a size of 1000 characters and an overlap of 200 characters to preserve contextual continuity between consecutive chunks. Each query is mapped to all chunks containing any part of the ground-truth answer, including cross-boundary spans. Coverage is the proportion of queries with at least one relevant chunk in the top- k retrieved results. Once chunking is complete next step is to create semantic representations of the chunks. The semantic representation is created using OpenAI's text-embedding-3-large model. The text-embedding-3-large model can create up to 3072-dimensional embeddings. At the end of this process, high-dimensional, dense embedding vectors that capture contextual and semantic content are created.

B. Storing Vectors

After the data preprocessing step, where document chunks were converted to the embeddings, the next step is to store the resulting high-dimensional vectors in a vector database. Embeddings are stored in the vector database for efficient similarity-based retrieval. Each stored vector was associated with its corresponding doc_id and chunk_id, ensuring traceability and allowing alignment with ground truth answers during evaluation. In this study, we used the Milvus database. The Milvus database is used because it is a high-performance, open-source vector database. Milvus supports both dense and hybrid (dense + sparse) embeddings, which is why it is optimized for large-scale similarity search. Milvus performs fast approximate nearest neighbor (ANN) searches using indexing strategies such as HNSW, IVF_FLAT, and SCANN. In this study, we used HNSW. Milvus supports several distance metrics for search purposes, including cosine, inner product,

and L2. The focus of this study is the comparison of these distance metrics, keeping all other parameters constant [21].

C. Retrieval and Ranking

Once embedding vectors are stored in Milvus next step is retrieval and ranking. The retrieval operation is performed on a query q . The embedding of q was generated using the same embedding model as used for document vectors (text-embedding-3-large model). The embedding is represented as: $\mathbf{v}_q \in \mathbb{R}^d$. In this study, retrieval was performed using HNSW graphs, an approximate nearest neighbor search algorithm specifically optimized for high-dimensional spaces.

1) *HNSW Indexing*: HNSW constructs a multi-layer graph structure G , which is represented as $G = (V, E)$, where each node corresponds to a vector embedding and edges represent neighborhood connections. Each node is assigned to a maximum level $\ell \sim \text{Geometric}(p)$, creating a hierarchy of layers $\{L_0, L_1, \dots, L_m\}$, with L_m being the sparsest top layer and L_0 the densest base layer. Every search begins from an entry node at the highest layer and proceeds greedily, descending layer by layer to refine candidate neighbors [22]. This design achieves logarithmic complexity in practice, represented by equation 1

$$\mathcal{O}(\log N) \quad (1)$$

In equation 1 N is the number of stored vectors.

a) *Distance Metrics*.: To quantify similarity between document and query embeddings, we evaluated three widely adopted distance or similarity measures (Cosine Similarity, Euclidean Distance, and Inner Products) [23]. These metrics are used to compare embeddings in HNSW graphs.

1. Cosine Similarity: Cosine similarity measures the cosine of the angle between two vectors in the embedding space. It is represented by equation 2

$$\text{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \quad (2)$$

2. Euclidean (L2) Distance: Euclidean distance, also known as ℓ_2 norm, measures the straight line distance between two points in the embedding space. It is shown by equation 3

$$d_{L2}(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^d (u_i - v_i)^2} \quad (3)$$

3. Inner Product: The inner product measures similarity through the direct multiplication and summation of corresponding vector components. It is represented by 4

$$\text{sim}_{\text{ip}}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^d u_i v_i \quad (4)$$

For retrieval, HNSW approximated the nearest neighbors of $\mathbf{v}_q$ by minimizing distance or maximizing similarity according to the chosen metric.

Three similarity conditions are used in this study: cosine similarity (unit-normalized), inner-product similarity (unit-normalized), and L2 distance on unit-normalized embeddings.

We have left raw (un-normalized) L2 distance to isolate the effect of vector scale.

D. ReRanking

Once retrieval is done next step is ranking the retrieved documents. To rank, we need a query vector $\mathbf{v}_q$. Given a query vector $\mathbf{v}_q$ and a set of document vectors $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ retrieved from the index, we ranked candidates using equation 5.

$$\text{rank}(d_i) = f(\mathbf{v}_q, \mathbf{v}_i), \quad \forall i \in \{1, \dots, k\} \quad (5)$$

In equation 5 $f(\mathbf{v}_q, \mathbf{v}_i)$ denotes the selected similarity function. After the initial similarity-based ranking, we further refined the order using a neural reranker, specifically the `cross-encoder/ms-marco-MiniLM-L-6-v2` model. This reranker jointly evaluates the query document pair and assigns a relevance score, thereby improving ranking quality beyond raw embedding similarity. In the end, the top k ranked candidates were passed to the generative model as retrieval augmented context.

E. Question Answering

After the final reranking step, the top k most relevant documents were passed as context to the large language model. In this study, we used the `gpt-4o-mini` model. The model received both the query and the selected context passages, enabling retrieval augmented generation (RAG) for producing grounded and contextually accurate answers.

F. Evaluation

The performance of our proposed RAG pipeline is evaluated using three types of evaluation metrics (Retrieval Metrics, Reranking Metrics, and Completion Metrics).

1) *Retrieval and Reranking Metrics*: The performance of the retrieval and reranking stage is evaluated using the following metrics.

- Recall@k (R@k): Recall@k (R@k) indicates the fraction of queries with at least one relevant document in the top k results;
- Mean Reciprocal Rank (MRR): Mean Reciprocal Rank (MRR) reflects the average rank position of the first relevant document;
- NDCG@k: NDCG weights relevant documents higher when appearing earlier in the ranked list.
- Coverage: Coverage represents the proportion of queries for which any relevant document is retrieved.

2) *Completion Metrics*: Once the top-ranked document chunks are identified, they are provided as context to a generative language model for answer generation. The evaluation of this stage focuses on:

- Latency: Time taken by the model to generate answers for a given query.
- Token Usage: Number of prompt and completion tokens consumed, which also relates to computational and financial cost when using API based models.
- Cost: Monetary cost of API calls per query (if applicable), derived from token usage.

TABLE I
RETRIEVAL AND RERANKING METRICS ACROSS DISTANCE METRICS

Stage	Metric	L2	Inner Product	Cosine
Retrieval	R@1	0.047	0.719	0.707
	R@5	0.193	0.883	0.877
	R@10	0.373	0.925	0.909
	MRR	0.116	0.793	0.781
	nDCG@1	0.047	0.719	0.707
	nDCG@5	0.116	0.811	0.802
	nDCG@10	0.175	0.825	0.812
	Coverage	0.373	0.925	0.909
Reranking	R@1	0.337	0.806	0.796
	R@5	0.370	0.912	0.897
	R@10	0.373	0.925	0.909
	MRR	0.351	0.852	0.840
	nDCG@1	0.337	0.806	0.796
	nDCG@5	0.356	0.866	0.853
	nDCG@10	0.357	0.870	0.857
	Coverage	0.373	0.925	0.909

IV. RESULTS

Table I summarizes the performance of the retrieval and reranking stages across three different distance metrics: L2, Inner Product, and Cosine. For the retrieval stage, L2 exhibits very low performance, with $R@1 = 0.0467$, $R@5 = 0.1925$, and $R@10 = 0.3733$, while Inner Product achieves significantly higher recall values of $R@1 = 0.7192$, $R@5 = 0.8825$, and $R@10 = 0.925$. Cosine is similarly strong with $R@1 = 0.7067$, $R@5 = 0.8767$, and $R@10 = 0.9092$. Mean Reciprocal Rank (MRR) also follows the same trend, with L2 at 0.1159, Inner Product at 0.7930, and Cosine at 0.7806. The nDCG scores and Coverage show similar patterns, highlighting that Inner Product and Cosine consistently retrieve more relevant documents than L2.

In the reranking stage, L2 improves modestly ($R@1 = 0.3367$, $R@5 = 0.37$, $R@10 = 0.3733$), whereas Inner Product ($R@1 = 0.8058$, $R@5 = 0.9117$, $R@10 = 0.925$) and Cosine ($R@1 = 0.7958$, $R@5 = 0.8967$, $R@10 = 0.9092$) maintain superior performance. MRR and nDCG values reinforce these observations, with L2 lagging behind (MRR = 0.3512) compared to Inner Product (0.8518) and Cosine (0.8395). Coverage indicates that Inner Product and Cosine consistently ensure at least one relevant document is retrieved for 92.5% and 90.9% of queries, respectively, compared to only 37.3% for L2. The Coverage metric remained identical for both the retrieval and reranking stages because it measures the proportion of queries for which at least one relevant item is retrieved, which is determined by the initial retrieval set; reranking only changes the order of items, not the set of retrieved candidates. Overall, these results demonstrate that both retrieval and reranking stages are highly sensitive to the choice of distance metric, with Inner Product and Cosine providing substantially better effectiveness than L2.

Table II shows the evaluation of the answer generation stage across L2, Inner Product, and Cosine metrics. Average query cost is similar across metrics (\$0.000272–\$0.000277), with prompt tokens ranging from 1,678.9 to 1,713.8 and completion

TABLE II
ANSWER GENERATION (COMPLETION) METRICS

Metric	L2	Inner Product	Cosine
Average Price	0.000277	0.000272	0.000273
Average Prompt Tokens	1,713.8	1,678.9	1,688.6
Average Completion Tokens	34.01	33.17	33.73
Average Total Tokens	1,747.81	1,712.10	1,722.33

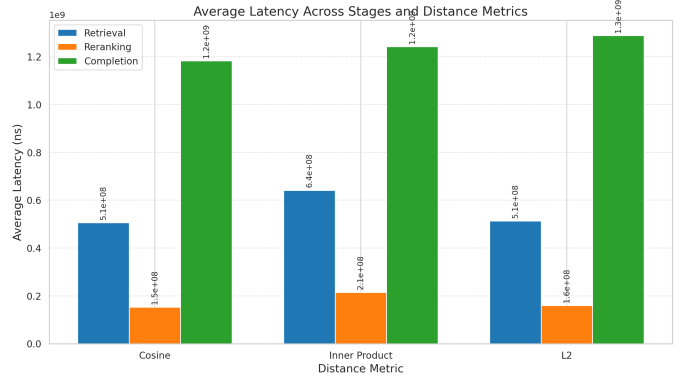


Fig. 2. Average Latency (ns) for Retrieval, Reranking, and Completion stages across different distance metrics.

tokens from 33.17 to 34.01, yielding total tokens between 1,712.10 and 1,747.81. These results indicate that the choice of distance metric has a minimal impact on completion, with token usage and cost remaining largely consistent. Figure 2 illustrates the average latency (in nanoseconds) for the three main stages of the system retrieval, reranking, and answer generation (completion) across three distance metrics: Cosine, Inner Product, and L2. Retrieval latency is lowest for Cosine (5.06×10^8 ns) and slightly higher for L2 and Inner Product. Reranking latency follows a similar trend, with Cosine again being the fastest (1.53×10^8 ns). Completion latency is orders of magnitude higher than the other stages, ranging from 1.18×10^9 ns for L2 to 1.29×10^9 ns for Cosine, indicating that answer generation dominates the total processing time. Overall, the figure highlights that while distance metric choice significantly affects retrieval and reranking performance, the completion stage remains the primary contributor to overall latency.

V. CONCLUSION

This study systematically examined the impact of similarity metrics on the performance of RAG pipelines. By evaluating Cosine, Inner Product, and L2 distance under identical conditions, we demonstrated that retrieval and reranking effectiveness is highly sensitive to metric choice, with Cosine and Inner Product consistently outperforming L2 across recall, MRR, nDCG, and coverage. In contrast, completion metrics, including token usage, cost, and latency, showed minimal variation across metrics, with answer generation remaining the dominant contributor to total processing time. These results underscore the importance of selecting appropriate similarity functions when deploying RAG systems, as retrieval quality

directly influences the relevance and grounding of generated answers.

A limitation of this study is that it evaluates the RAG pipeline on a single dataset with fixed HNSW indexing. While Cosine, Inner Product, and L2 were compared, the effects of dataset size, alternative indexing strategies, and different generative models remain unexplored.

In future work, we will extend this work to include raw L2 embeddings, as well as additional public benchmarks such as BEIR, with varying task characteristics and enterprise style corpora containing long, heterogeneous documents. We also plan to use more diverse vector indexing strategies (e.g., IVF_FLAT, SCANN), and multiple generative models to provide a comprehensive comparative analysis of an RAG pipeline.

Acknowledgement: This study is part of a thesis titled "Performance Analysis of Advanced Retrieval-Augmented Generation Applications using Vector Databases, Indexing Algorithms and Distance Metrics" under the supervision of Jawad Rasheed.

REFERENCES

- [1] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen *et al.*, "Siren's song in the ai ocean: A survey on hallucination in large language models," *Computational Linguistics*, pp. 1–46, 2025.
- [2] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," 2024.
- [3] A. Balaguer, V. Benara, R. L. d. F. Cunha, T. Hendry, D. Holstein, J. Marsman, N. Mecklenburg, S. Malvar, L. O. Nunes, R. Padilha *et al.*, "Rag vs fine-tuning: pipelines, tradeoffs, and a case study on agriculture," *arXiv preprint arXiv:2401.08406*, 2024.
- [4] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity," *arXiv preprint arXiv:2403.14403*, 2024.
- [5] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven failure points when engineering a retrieval augmented generation system," in *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, 2024, pp. 194–199.
- [6] S. Sivasothy, S. Barnett, S. Kurniawan, Z. Rasool, and R. Vasa, "Ragprobe: Breaking rag pipelines with evaluation scenarios," in *2025 IEEE/ACM 4th International Conference on AI Engineering-Software Engineering for AI (CAIN)*. IEEE, 2025, pp. 60–71.
- [7] I. L. Bessas, F. L. C. Pádua, G. T. de Assis, R. T. N. Cardoso, and A. Lacerda, "Automatic and online setting of similarity thresholds in content-based visual information retrieval problems," *EURASIP Journal on Advances in Signal Processing*, vol. 2016, no. 1, p. 32, 2016.
- [8] S. Roychowdhury, S. Soman, H. Ranjani, N. Gunda, V. Chhabra, and S. K. Bala, "Evaluation of rag metrics for question answering in the telecom domain," *arXiv preprint arXiv:2407.12873*, 2024.
- [9] G. Şahin, K. Varol, and B. K. Pak, "Llm and rag-based question answering assistant for enterprise knowledge management," in *2024 9th International Conference on Computer Science and Engineering (UBMK)*. IEEE, 2024, pp. 1–6.
- [10] J. Hu, Y. Zhou, and J. Wang, "Intrinsic evaluation of rag systems for deep-logic questions," *arXiv preprint arXiv:2410.02932*, 2024.
- [11] K. Roy, Y. Zi, C. Shyalika, R. Prasad, S. Murali, V. Palit, and A. Sheth, "Qa-rag: Leveraging question and answer-based retrieved chunk re-formatting for improving response quality during retrieval-augmented generation," 2024.
- [12] B. Saha, U. Saha, and M. Z. Malik, "Advancing retrieval-augmented generation with inverted question matching for enhanced qa performance," *IEEE Access*, 2024.
- [13] M. Barochiya, P. Makhijani, H. N. Patel, P. Goel, and B. Patel, "Evaluating rag pipeline in multimodal llm-based question answering systems," in *2024 3rd International Conference on Automation, Computing and Renewable Systems (ICACRS)*. IEEE, 2024, pp. 69–75.
- [14] S. Veturi, S. Vaichal, R. L. Jagadheesh, N. I. Tripto, and N. Yan, "Rag based question-answering for contextual response prediction system," *arXiv preprint arXiv:2409.03708*, 2024.
- [15] A. Alinejad, K. Kumar, and A. Vahdat, "Evaluating the retrieval component in llm-based question answering systems," *arXiv preprint arXiv:2406.06458*, 2024.
- [16] K. Zhu, Y. Luo, D. Xu, Y. Yan, Z. Liu, S. Yu, R. Wang, S. Wang, Y. Li, N. Zhang *et al.*, "Rageval: Scenario specific rag evaluation dataset generation framework," *arXiv preprint arXiv:2408.01262*, 2024.
- [17] M. Špeletić, S. Cvetković, M. Protić, and S. V. Nikolić, "Exploring rag in medical question answering: Integrating llms and vector databases," 2024.
- [18] Z. Qiao, M. Gao, H. Ma, X. Wen, E. Yan, W. Zhao, H. Cao, and H. Wu, "Intelligent question answering system for power regulations based on rag," in *2024 4th International Conference on New Energy and Power Engineering (ICNEPE)*. IEEE, 2024, pp. 1155–1161.
- [19] K. Muludi, K. M. Fitria, J. Triloka *et al.*, "Retrieval-augmented generation approach: Document question answering using large language model," *International Journal of Advanced Computer Science & Applications*, vol. 15, no. 3, 2024.
- [20] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
- [21] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 international conference on management of data*, 2021, pp. 2614–2627.
- [22] R. Masuda, K. Iwamoto, K. Ando, and H. Kamei, "Tiered cache-hnsw: Using hierarchical caching system in hnsw," in *2025 1st International Conference on Consumer Technology (ICCT-Pacific)*. IEEE, 2025, pp. 1–4.
- [23] S. Trangcasanchai, "Improving question answering systems with retrieval augmented generation," Ph.D. dissertation, University of Helsinki, 2024.